

Taller Práctico 4: **Red neuronal en una GPU**

Estudiantes: Kevin García, Juan Márquez, Karol Guerrero
Universidad Sergio Arboleda, abril 2026

1. Objetivo del laboratorio:

Este taller tiene como objetivo recrear una red neuronal utilizando tarjetas gráficas, mediante la implementación de CUDA en lenguaje python, para esto se deberá extraer los parámetros de un modelo de red neuronal ya entrenado para clasificar imágenes de dígitos numéricos.

2. Marco teórico:

El procesamiento de video consiste en analizar un video como una secuencia de imágenes que se conoce como frames y esto se realiza con el fin de poder identificar cambios o detectar objetos en movimiento, es decir que en lugar de ver el video completo el algoritmo trabaja comparando cada frame con el anterior para encontrar qué partes cambian con el tiempo.

En este laboratorio, el procesamiento de video se enfoca en transformar cada frame de color a escala de grises. Esto implica modificar la representación de los píxeles, pasando de un modelo de color RGB (Rojo, Verde y Azul) a un único valor de intensidad. Esta conversión reduce la complejidad de los datos ya que, en lugar de manejar tres canales de color se trabaja con uno solo.

El proceso matemático común para esta transformación consiste en calcular un promedio ponderado de los valores RGB de cada píxel, considerando que el ojo humano percibe con mayor sensibilidad el color verde. De esta manera, se obtiene un valor de gris que representa adecuadamente la luminancia de la imagen.

Además, en este laboratorio se implementa el procesamiento utilizando GPU mediante CUDA, lo que permite realizar operaciones en paralelo sobre grandes cantidades de píxeles. Esto mejora significativamente el rendimiento en comparación con el procesamiento secuencial en CPU, ya que múltiples hilos pueden trabajar simultáneamente en diferentes partes del video.

3. Metodología

a. Configuración del Hardware

Para el desarrollo del laboratorio se utilizó un entorno de ejecución en Google Colab y se utilizó una GPU NVIDIA Tesla T4, con soporte para CUDA 13.0 lo que permitió ejecutar algoritmos de manera paralela. Esta GPU cuenta con aproximadamente 15 GB de memoria, el uso de la GPU fue verificado mediante TensorFlow, confirmando que el dispositivo estaba disponible para acelerar las operaciones.

b. Configuración del Software

El entorno de desarrollo se configuró con el lenguaje de programación Python, se emplearon las siguientes librerías y versiones:

- **TensorFlow 2.19.0**, utilizado para la construcción y entrenamiento de la red neuronal
- **Keras**, como API de alto nivel para definir el modelo
- **NumPy**, para operaciones matemáticas y manipulación de datos
- **Matplotlib**, para visualización
- **os**, para manejo básico del sistema
- **GPU detectada:** /physical_device:GPU:0

c. Explicación del algoritmo desarrollado

Para el desarrollo del algoritmo se utilizó el dataset MNIST, el cual está compuesto por 60,000 imágenes para entrenamiento y 10,000 para prueba. Estas imágenes corresponden a dígitos escritos a mano (del 0 al 9) y tienen un tamaño de 28x28 píxeles en escala de grises, donde cada píxel representa un nivel de intensidad.

Antes del entrenamiento, se realizó un proceso de preprocesamiento de los datos. Originalmente, los valores de los píxeles se encuentran en un rango de 0 a 255, por lo que se aplicó una normalización dividiendo cada valor entre 255. Esto permitió transformar los datos a un rango entre 0 y 1 lo cual mejora la estabilidad del modelo y facilita el proceso de aprendizaje.

Luego, se implementó una red neuronal tipo Perceptrón Multicapa (MLP). La estructura del modelo está compuesta por una capa de entrada, varias capas ocultas y una capa de salida, organizadas de la siguiente manera:

- **Capa de entrada:** se realiza el aplanamiento de la imagen, transformando la matriz de 28x28 en un vector de 784 valores.
- **Capas ocultas:** se utilizan cuatro capas densas, cada una con 128 neuronas y función de activación ReLU, encargadas de extraer características relevantes de los datos.
- **Capa de salida:** está compuesta por 10 neuronas (una por cada clase) y utiliza la función de activación Softmax para generar probabilidades.

El modelo fue entrenado utilizando el optimizador Adam, la función de pérdida *sparse categorical crossentropy* y la métrica de accuracy para evaluar el desempeño. El entrenamiento se llevó a cabo durante varias épocas utilizando el conjunto de datos de entrenamiento y finalmente, una vez entrenado el modelo se extrajeron los parámetros aprendidos, específicamente los pesos (W) y los bias (b) de las capas ocultas.

4. Resultados:

A continuación, se presentan los resultados obtenidos durante la ejecución del laboratorio. En primer lugar, se muestran las dimensiones de los pesos y bias extraídos de cada una de las capas de la red neuronal.

Capa	Dimensión de Pesos (W)	Dimensión de Bias (b)
Capa 1	(784, 128)	(128,)
Capa 2	(128, 128)	(128,)
Capa 3	(128, 128)	(128,)
Capa 4	(128, 128)	(128,)
Capa 5	(128, 10)	(10,)

Tabla 1. Pesos y bias por capa.

En la siguiente figura se presenta el comportamiento del modelo durante el entrenamiento, donde se muestra la evolución de la precisión y la pérdida a lo largo de las épocas.

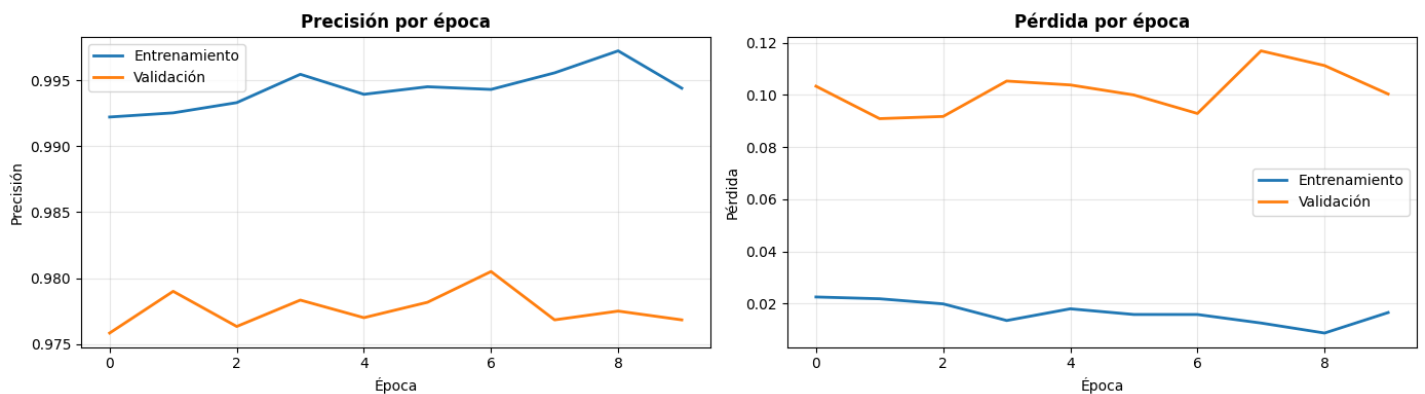


Imagen 1. Precisión y pérdida por época

Se realizó una prueba de predicción utilizando una imagen de entrada. El modelo generó una predicción junto con la distribución de probabilidades obtenida mediante la función Softmax

Tu dígito → Predicción: 6

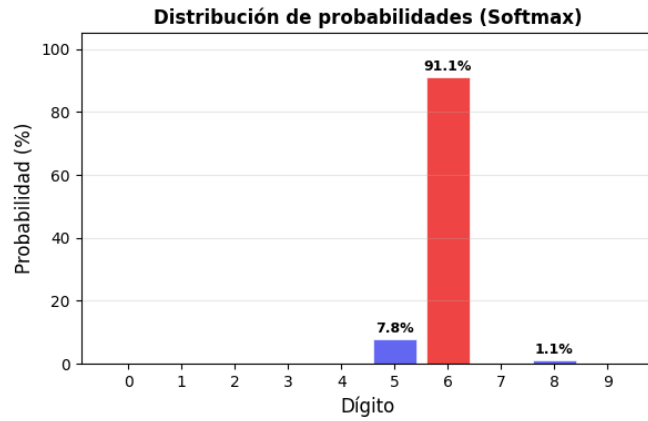
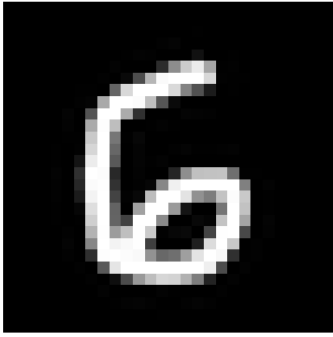


Imagen 2. Prueba de predicción

Dígito	Probabilidad
0	0.1%
1	0.0%
2	0.0%
3	0.0%
4	0.0%
5	7.8%
6	91.1% (Predicción)
7	0.0%
8	1.1%
9	0.0%

Tabla 2. Probabilidades por dígito.

Se realizó una comparación de rendimiento entre CPU y GPU para el proceso de *forward propagation*. Se registraron los tiempos de ejecución para diferentes tamaños de lote en ambas implementaciones.

Batch	CPU (ms)	GPU (ms)	Speedup
1	0.30	1.23	0.24x
10	1.84	1.29	1.42x
100	1.88	2.02	0.93x
500	5.12	3.90	1.31x
1000	9.22	8.12	1.14x

Tabla 3. Comparación de rendimiento CPU vs GPU

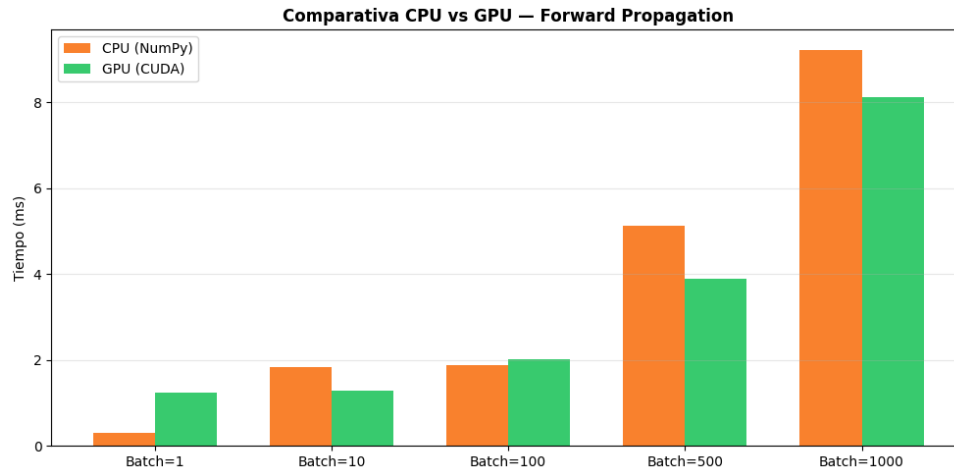


Imagen 3. CPU vs GPU

Se presentan ejemplos de predicciones realizadas por la red implementada en CUDA sobre imágenes de prueba, mostrando la clase predicha y la etiqueta real correspondiente para cada caso.



Imagen 4. Ejemplos de predicciones

5. Análisis del rendimiento:

A partir de los resultados obtenidos, se puede hacer una comparación entre la implementación de la red neuronal en CPU utilizando Keras y su reconstrucción en GPU mediante CUDA, donde en términos de precisión ambas alcanzaron un valor de 97.70%, lo que permite ver que la implementación manual logró replicar correctamente el comportamiento del modelo original.

En cuanto a los tiempos de ejecución, se observa una diferencia bastante grande ya que la versión en CUDA registró un tiempo de 8.1 ms, mientras que la implementación en Keras presentó un tiempo de 1343.3 ms, lo que muestra claramente cómo el uso de la GPU permite reducir en gran medida el tiempo de ejecución.

Al analizar los resultados según el tamaño del batch se puede notar que la GPU no siempre es más rápida en todos los casos, por ejemplo cuando el batch es muy pequeño como en batch = 1 la CPU obtiene mejores tiempos, esto se puede deber a la sobrecarga inicial que tiene la GPU, sin embargo a medida que el tamaño del batch aumenta como en 10, 500 y 1000, la GPU empieza a mostrar un mejor rendimiento aprovechando su capacidad de procesamiento en paralelo.

Por otro lado, en el procesamiento de video el uso de la GPU permite trabajar múltiples píxeles al mismo tiempo, lo que reduce el tiempo necesario para transformar los frames a escala de grises, haciendo que este tipo de tareas sea mucho más eficiente cuando se utiliza paralelismo.

Finalmente, en las pruebas de predicción se observa que el modelo logra identificar correctamente los dígitos de entrada, asignando una mayor probabilidad a la clase correcta en cada caso, lo que indica que tanto el entrenamiento como la implementación del algoritmo funcionaron de manera adecuada.

Conclusiones:

En este laboratorio se logró entender mejor el funcionamiento de una red neuronal, especialmente cómo a partir de operaciones matemáticas como multiplicaciones, suma de bias y funciones de activación se pueden generar predicciones. También fue importante ver que el modelo no es una “caja negra”, ya que al extraer los pesos y bias se pudo replicar manualmente la red, obteniendo los mismos resultados que con Keras, lo que ayuda a comprender realmente cómo se hace la clasificación.

Por otro lado, al ejecutar la red en GPU se observó que este tipo de modelos se benefician bastante del paralelismo, ya que muchas operaciones se pueden hacer al mismo tiempo, reduciendo los tiempos de ejecución.